```
import { q, j } from '../db.js';

const subscribers = new Map(); // topic -> [fn]

export function subscribe(topic, fn) {
  if (!subscribers.has(topic)) subscribers.set(topic, []);
  subscribers.get(topic).push(fn);
}

// Written inside the same transaction as the change that caused it.
export async function emit({ businessId = null, topic, payload = {} }) {
  await q(
    `insert into event_outbox (business_id, topic, payload) values ($1,$2,$3)`,
    [businessId, topic, JSON.stringify(payload)]
  );
}

// Drain the outbox and hand each event to its subscribers.
export async function drain({ limit = 100 } = {}) {
  const rows = await q(
    `select * from event_outbox where delivered_at is null order by id limit ${Nu
  );
  for (const row of rows) {
    const payload = j(row.payload);
    for (const fn of subscribers.get(row.topic) ?? []) {
      try { await fn({ businessId: row.business_id, topic: row.topic, payload });
      catch (e) { console.error(`subscriber failed on ${row.topic}:`, e.message);
    }
    await q(`update event_outbox set delivered_at = now() where id = $1`, [row.id
  }
  return rows.length;
}
```